Developer Challenges in the Adoption of Model Context Protocol in AI Agent Development

Oluwatoyosi Oyelayo
oluwatoyosi.oyelayo@mail.concordia.ca
Concordia University
Montreal, QC, Canada

Samuel Abedu
samuel.abedu@mail.concordia.ca
Concordia University
Montreal, QC, Canada

SayedHassan Khatoonabadi
sayedhassan.khatoonabadi@concordia.ca
Concordia University
Montreal, QC, Canada

Emad Shihab
emad.shihab@concordia.ca
Concordia University
Montreal, QC, Canada

Abstract

Advances in the field of artificial intelligence have led to the introduction of autonomous applications (AI Agents), capable of interacting with external application interfaces. While these AI agents have seen increased adoption, a recurring challenge is a standard communication mechanism between these agents and external applications. The Model Context Protocol (MCP) was introduced as a standardized interface to address this challenge. However, little is known about practical issues experienced by developers in its adoption. Our goal for this study is to identify the challenges faced by developers in the adoption of MCP and the effort expended in addressing them. To achieve this, we analyze bug reports and pull requests on MCP reference servers. Our findings reveal that issues relating to file system operations (21.32%) are the most prevalent, followed by data validation and type issues (13.97%). Also, we find that developers spend more effort addressing connection and communication issues. Our work presents the MCP issues developers encounter and provides insight into the extent of work required to resolve them. Our findings can provide MCP repository maintainers and contributors with a structured view of developer issues and help them make informed prioritizations.

CCS Concepts

• **Software and its engineering → Maintaining software**.

Keywords

Model Context Protocol, LLMs, Agentic AI

ACM Reference Format:
Oluwatoyosi Oyelayo, Samuel Abedu, SayedHassan Khatoonabadi, and Emad Shihab. 2025. Developer Challenges in the Adoption of Model Context Protocol in AI Agent Development. In *Proceedings of the 7th International Workshop on Bots and Agents in Software Engineering (BoatSE '26).* ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
BoatSE '26, Rio De Janeiro, Brazil
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The field of artificial intelligence has seen remarkable growth over the years, resulting in the development of autonomous agents. These agents have seen increased practical use cases in diverse real-world domains [14] such as finance [5, 17], healthcare [5, 15], travel [16] and software development [6], where they have been applied to automate complex tasks. These advances have been driven by the integration of large language models (LLM) with external application interfaces, enabling agents to leverage tools to perform a wide range of operations [7].

A recurring challenge with the development of these applications, is a seamless and standardized access to external data and tools [8]. To address this, major model providers introduced tool calling [5] - a mechanism that allows language models to interact with external systems. Yet, diverse AI agent frameworks and language models have distinct interfaces and integration methods, resulting in the introduction of significant integration complexities across platforms. To address this challenge, Anthropic introduced the Model Context Protocol (MCP) - a standardized open-source interface for connecting AI applications to external systems, including data sources such as files and databases, and tools such as search engines[1].

MCP has received wide adoption since its release in late 2024 [7], and its reference servers have become central to the ecosystem. These servers were developed by Anthropic with the aim of demonstrating how to connect AI applications to external data sources and tools using the protocol, and to serve as a guide for the development of other servers[2] [1]. While MCP addresses the existing challenge of easing the process of providing context to language models, it still has its own drawbacks. Prior research emphasized maintainability and security considerations [5, 7] as the major drawbacks of MCP. However, little is known about the challenges developers face on a day-to-day basis when integrating MCP in practice. Therefore, in this study, our goal is to identify the challenges developers encounter when integrating MCP in their workflow and evaluate the extent of work required to address these challenges. To achieve this goal, we answer two research questions:

RQ1: **What are the common challenges in the integration of MCP?** We manually analyze developer-reported issues on MCP reference servers to identify these challenges. We find that **file**

[1] https://modelcontextprotocol.io/docs/getting-started/intro
[2] https://www.anthropic.com/news/model-context-protocol

path and file system issues are the most common (21.32%), majorly affecting the FileSystem MCP server. The second and third most common challenges faced by developers are **data validation and type issues** (13.97%) and **tool invocation, visibility, and behavior issues** (13.24%), respectively.

RQ2: **What is the extent of work involved in resolving issues affecting MCP server integration?** We estimate the extent of work involved in resolving these issues using code churn as the metric [4]. We find that **connection and communication issues require the highest level of work to resolve (1143.5)**. This is followed by file path and file system issues (492.0), and data validation and type issues (402.7).

Contributions. Our study makes the following contributions:

(1) We provide a structured categorization of developer-reported issues on the official MCP repository.
(2) We provide insight into the level of work required to resolve the various categories of issues.
(3) Since the MCP repository does not have any visible strategy for maintainers to prioritize reported issues, we provide insight into the categories of issues that should be prioritized more.
(4) To promote the reproducibility of our study and facilitate future research on this topic, we publicly share our scripts and dataset online [9]

Paper Organization. The rest of the paper is organized as follows. Section 2 describes related works. Section 3 describes our study design. Section 4 presents the results of our two research questions. Section 5 discusses key points from our findings. Section 6 outlines the threats to the validity of our study. Section 8 concludes this paper.

2 Related Works

In this section, we review existing studies related to the challenges and concerns in the adoption of MCP. Existing studies have explored these challenges [3, 5, 7, 10, 13] only from security and maintainability perspectives. Hou et al. [7] regard MCP as an open standard that specifies a unified, bi-directional communication and dynamic discovery protocol between AI models and external tools or resources that enhance interoperability and reduce fragmentation across diverse systems. They focus on security concerns by creating a threat taxonomy that comprises four major attack types: malicious developers, external attackers, malicious users, and security flaws. To validate these risks, the authors develop and analyze real-world cases that demonstrate attack and vulnerability types with MCP implementations and propose a set of actionable steps for a secure MCP adoption.

Hasan et al. [5] perform an empirical study of official and community MCP servers. Focusing on health metrics and a hybrid analysis pipeline, the authors evaluate 1,899 open-source MCP servers, using static code analysis tools. The authors identify eight vulnerability types - credential exposure, lack of access control, CORS issues, improper resource management, transport security issues, authentication issues, insecure file creation, and input validation issues. The authors also evaluate how healthy and sustainable MCP servers are, using 14 repository-level metrics from existing literature, and find that they demonstrate higher or equal median values in 9 out of 14 key metrics in comparison to OSS and ML baselines, concluding on promising sustainability.

More recently, Tiwari et al. [13] perform an audit of MCP in vision systems, identifying weaknesses in schema semantics, interoperability, and runtime coordination by analyzing 91 publicly registered vision-related MCP servers to identify failures and categorize them. The authors define the following failure modes: schema divergence and interface ambiguity, lack of runtime schema validation, composition failures and bridging scripts, and the absence of evaluation frameworks.

Existing research has primarily focused on static code analysis to identify issues such as security vulnerabilities and maintainability issues; our research focuses on self-reported challenges by developers in the adoption of MCP servers. In our study, we go beyond theoretical analysis to gain insight directly from developer reports. Hence, providing a structured categorization of issue types, resolution insights, and recommendations on how to mitigate these challenges.

3 Study Design

The goal of our study is to identify the challenges faced by developers in the integration of MCP and evaluate the level of work required in resolving them. In this section, we describe the dataset used in our study. We present our approach for the dataset curation in Figure 1.

3.1 Dataset Collection and Filtering

To understand the challenges developers face when integrating MCP servers, we analyze issues from the official MCP servers repository [1]. We focus exclusively on the seven reference servers. Table 1 presents the description of each of the reference servers. We focus on the reference servers because these were built by Anthropic, the protocol's authors, and they therefore adhere closely to the official specification. Limiting our scope to them reduces noise from third-party misinterpretation of the specification, non-adherence to the specification, or poor code quality, and provides a consistent baseline for assessing the protocol's maturity.

Using the GitHub API, we collect all 684 issues in the MCP official GitHub repository as of October 10, 2025. We filter the issues labeled "bug", yielding **309 issues**. We include both open and closed issues, as our goal is to categorize all issues encountered in the integration of MCP, regardless of whether they have been fixed or not.

To ensure that our dataset contains only issues related to reference servers, we apply a keyword-based filter implemented in Python that searches for specific substrings associated with reference servers as well as patterns that frequently appear in MCP server installation. Our filtering approach assumes that any issue without a mention of a reference server-related term or keyword (e.g., git-mcp, fileserver, filesystem-mcp) falls outside the scope of our research. We employ keyword-based filtering because it is widely used in empirical software development research, and it provides an efficient and reproducible approach to cleaning data [11, 12]. For example, to identify issues relating to the Fetch MCP server, we use keywords such as 'fetch-mcp', 'fetch mcp', 'fetch-server', 'fetch server', 'mcp fetch', 'mcp-fetch', 'server/fetch', 'server fetch', 'server-fetch', 'mcp/fetch', 'mcp-server-fetch', and

Table 1: MCP Reference Servers and their Descriptions

Server	Description
Everything	This is a test server aimed at experimenting with MCP. It implements prompts, tools, and resources.
Fetch	Enables LLMs to retrieve and process content from web pages and convert them to markdown.
Filesystem	Enables file system operations with configurable access controls.
Git	Provides LLMs with tools to read, search, and manipulate Git repositories.
Memory	Provides persistent memory using a local knowledge graph, enabling the AI agent to remember information about the user across chats.
Sequential Thinking	Provides a dynamic and reflective problem-solving tool through a structured thinking process.
Time	Provides tools to get current time information and perform timezone conversions.

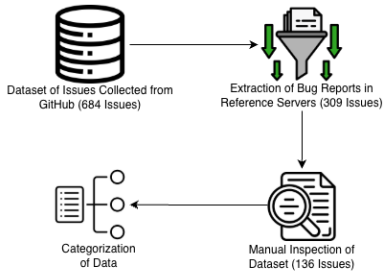

Figure 1: Overview of our approach for collecting, analyzing, and categorizing developer-reported MCP issues

'mcp_server_fetch'. For an MCP server like Fetch, we intentionally avoid using the standalone keyword "fetch", as it could appear in unrelated contexts (e.g., Javascript fetch() calls). Similarly, for the Git reference server, we carefully construct our keywords by including spaces after the keyword "git " to exclude third-party servers like GitHub and GitLab, which are outside the scope of this study. **After applying our filters, our dataset was reduced to a total of 153 issues**.

3.2 Dataset Validation

To confirm the validity of our automated filtering, the first two authors independently conduct a manual review of all 153 issues to filter out any leftover non-reference servers. Any discrepancies and disagreements in the filtering are resolved through discussion. After this process, we identify 17 false positives - issues incorrectly classified by the script as reference server-related and filter them out. These false positives account for approximately 11% of the dataset. The manual validation process is to ensure that our final dataset of **136 issues** accurately reflects reference server issues.

4 Results

In this section, we present the results of our two research questions. For each research question, we present its motivation, the approach to answer the question, and the results.

RQ1: What are the common challenges in the integration of MCP?

Motivation. Despite MCP gaining popularity in the standardization of how LLMs interact with external data sources and tools [5], there exists no empirical data of developer-reported challenges in its adoption. Existing studies only focus on security vulnerabilities and static code analysis [5, 7]. Thus, in this RQ, we aim to identify and categorize these challenges as reported by developers who have experienced issues integrating these servers. This will provide insights to the repository maintainers and contributors on the different prevalent issues reported by developers. Hence, guiding future improvements and reducing the challenges faced by developers.

Approach. To answer this research question, two authors manually review the title, body, and the discussion of the 136 issues in the dataset described in section 3 to identify recurring patterns and types of challenges reported by developers. The goal of the labeling is to categorize recurring patterns and types of challenges reported by developers. The first author performs an initial manual labeling of the issues to determine its primary cause, grouping issues by shared characteristics (e.g., area of occurrence in the MCP lifecycle and types of errors). This forms the basis for our structured categorization and serves as the initial labels. A second author independently uses the same labels, proposing additional labels where needed to manually categorize the issues. We compare both sets of labels and measure inter-rater agreement using Cohen's Kappa [2]. Our $\kappa = 0.68$ indicating substantial agreement. We then reconcile disagreements through discussion until consensus is reached, refining the categories and strengthening their validity.

Results. Table 2 shows the results of our manual classification of the MCP GitHub issues.

File Path and File System Issues (21.32%): These are issues pertaining to file system operations across various operating systems. Our analysis shows that this category accounts for the most reported issues by developers. A significant number of these issues involve path conversion and resolution across different operating systems, usually resulting in `ENOENT error` (file not found) or `access denied errors` (e.g. issue #2526). Several developers report cases where valid file paths are incorrectly resolved due to case sensitivity inconsistencies, relative path handling, or path validation (e.g. issue #1987). Other file system issues reported demonstrate problems encountered when file paths contain spaces, hyphens, or special characters, often resulting in failures (e.g issue #1770).

Additionally, developers report failures related to the creation of directories and access control. For example, there are reports about the inability to recursively create directories (e.g issue #711), access-denied errors for legitimate paths (e.g issue #470), and unexpected access to files outside a sandbox environment via symlinks (e.g. issue #1845), indicating a potential security vulnerability. Notably, there are reports of complete server crashes when configured paths

Table 2: Categories of Developer-Reported Issues

Category	Definition	Affected Servers	Frequency
File Path and File System Issues	Issues pertaining to file system operations across various operating systems.	filesystem, memory	29 (21.32%)
Data Validation and Type Issues	Issues involving incorrect data types, inadequate input validation, and wrong type checking.	everything, fetch, filesystem, memory, sequential-thinking	19 (13.97%)
Tool Invocation, Visibility, and Behavior Issues	Issues with tool discovery, invocation, and unexpected behavior during tool execution.	everything, filesystem, git, memory, sequential-thinking	18 (13.24%)
Configuration and Environment Issues	Issues with setup and configuration parameters.	All reference servers	17 (12.50%)
Documentation and Tool Description Gaps	Issues representing usage confusion as a result of inconsistent, inaccurate, outdated, or missing documentation or tool descriptions.	All reference servers.	16 (11.76%)
Connection and Communication Issues	Issues relating to unstable or failed connection between MCP clients and servers.	everything, fetch, filesystem, sequential-thinking, time	15 (11.03%)
Start up and Initialization Failures	Issues that prevent the MCP server from moving past the initialization phase.	fetch, filesystem, git, memory, sequential-thinking, time	10 (7.35%)
Timezone Issues	Issues pertaining to time zone handling and localization issues, particularly affecting the Time MCP server.	time	4 (2.94%)
Crashes and Runtime Errors	Issues of abrupt crashes, usually without an explicit or clear error message.	filesystem, sequential-thinking	3 (2.21%)
Build Errors	Issues that occur when developers attempt to build their application in which an MCP server is integrated.	filesystem, git	2 (1.47%)
Others	Issues include a progress notification issue, a missing resource issue, and an insufficient log report.	everything, memory	3 (2.22%)

are invalid (e.g. issue #2815), suggesting inadequate error handling logic. In summary, these findings show the need to ensure reliable and secure cross-platform file system operations. It is worth noting that the majority of these issues are specific to the FileSystem MCP server, suggesting that these challenges are primarily from the server's implementation rather than the protocol itself.

Data validation and Type Issues (13.97%): These are issues involving incorrect data types, inadequate input validation, and wrong type checking. These issues usually surface after successful server initialization. Specifically, a significant number of these issues pertain to schema compliance failures. For example, in issue #2473, the Sequential Thinking server frequently fails with "invalid totalThoughts: must be a number" when clients pass non-numeric values or improperly formatted parameters. Similarly, there are reports of parameter mismatches. Another case in this category is where Claude passes "undefined" instead of a properly structured object when making server calls (issue #2410), resulting in "invalid arguments" validation errors. Last in this category are reports pertaining to character encoding which particularly affects Windows users, where servers fail to handle non-ASCII characters properly (e.g. issue #2098).

Tool Invocation, Visibility, and Behavior Issues (13.24%): This category captures issues with tool discovery, invocation, and unexpected behavior during tool execution. Specifically, we capture issues relating to difficulty with AI agents identifying the right tool to use. There exists a report in issue #2818 about Everything server, where the client requests available tools using the standard tools/list method, but fails with "unable to read roots on an undefined property", suggesting an incomplete error handling in the tool listing process. There is also a persistent JSON parsing error in the Memory Server's "create_entities" and "create_relations" tools (issue #2689), indicating problems with data serialization. There is also a severe problem where the "git_add" tool incorrectly tracks "./git/" directory files, causing repository corruption (issue #2373).

Configuration and Environment Issues (12.50%): The configuration and environment category covers problems with setup and configuration parameters. Unlike the startup failures that prevent servers from launching, these issues involve servers that are able to start but are unable to operate correctly due to configuration problems or environment setup issues. Some developers report corruption to local environment files. Notable in this subset is a situation where a developer reports the Memory server's "memory.json" file getting corrupted due to concurrent write operations, resulting in JSON parsing errors (issue #2579). This corruption occurs when multiple processes attempt to modify the configuration file simultaneously, resulting in file corruption, which then requires manual update. Another developer reports an unexpected functionality caused by an out-of-date package. (issue #2431). In this scenario, the affected server lists fewer tools than documented, causing a confusion with understanding what the correct list is. Furthermore, there exists a report of a tool availability discrepancy due to a server being published under an incorrect label (issue #1908).

Documentation and Tool Description Gaps (11.76%): This category of issues represents usage confusion as a result of inconsistent, inaccurate, outdated, or missing documentation or tool descriptions. For example, in issue #2032, a user reports that the "read_multiple_files" tool of the FileSystem server lacks a clear description, causing AI assistants to use it incorrectly. Similarly, AI systems confuse the use of "search_files" and "search_file_contents" tools due to overlapping descriptions (issue #896). Also, there are reports of tools with missing documentation of optional parameters, default values, or parameter constraints, forcing users to resort

to trial-and-error approaches. Last in this category are reports of broken and non-existent reference links in the repository's documentation (e.g. issue #294). All of these issues highlight the need to pay attention to the documentation.

Connection and Communication Issues (11.03%): This category of issues captures reports relating to unstable or failed connections between MCP clients and servers. Developers report sudden disconnections or unexpected termination of connection after the initialization phase of the protocol. When this situation occurs, the MCP server unexpectedly exits instead of maintaining a persistent connection (e.g. #issue 2517). Common error messages reported are "Not Connected" and "Connection closed". Some reports also attribute these challenges with incompatibilities across various AI clients. For example, a developer reports in issue #2338 about connection problems with Claude. In summary, this category highlights the challenge of maintaining a stable client-server connection.

Start up and Initialization Failures (7.35%): These are issues that prevent the MCP server from moving past the initialization phase. They are critical issues, as they render the MCP server completely unusable. A significant number of these issues are a result of missing or incompatible dependencies. For example, a developer reports in issue #2421 that the Sequential Thinking MCP server fails to start in Cursor IDE with the error "Cannot find module...node_modules/zod/index.js". Another developer reports successfully building a Dockerfile to run the Fetch MCP server but unable to get a response from the initialization phase. Notable in this category is a report where calls to the Sequential Thinking server frequently hangs indefinitely (issue #2762). All of these issues highlight the critical nature of this category as it often makes it impossible to use the affected servers.

Timezone Issues (2.94%): This category of issues pertains to time zone handling and localization problems, particularly affecting the Time MCP server. These issues highlight challenges supporting users across different geographical locations, languages, and daylight saving time transitions. For example, in issue #786, a user reports that the server fails during daylight saving time transitions from EST to EDT, resulting in an error "No time zone found with key EDT". Additionally, a significant barrier is observed for international users with timezone names where the server is unable to map non-English timezone names to standard ones. For example, in issue #231, the reporter complains that the server fails when the time zone name is not in English.

Crashes and Runtime Errors (2.21%): For this category, we assign issues where there are reports of abrupt crashes, usually without an explicit or clear error message. For example, in issue #2792, a user reports that the Sequential Thinking server sometimes crashes, producing random errors even when such operations worked previously. This type of issues raises concerns about the reliability of these servers for production use cases.

Build Errors (1.47%): This category of issues captures challenges that occur when developers attempt to build their application in which an MCP server is integrated. Specifically, they occur as a result of dependency issues in specific MCP servers. For example, a user reports getting missing dependency errors after running "pnpm run -r build" in issue #1589. Another user reports a Docker build failure due to version inconsistency in "uv.lock" in issue #997. While it is unclear if these are issues with the implementation approach by the user or MCP itself, it is worth stating for completeness of our result.

Others (2.22%): We categorize three issue types as "others" since they appear only once, each in our dataset. They include progress notification, missing resource, and insufficient logs issues. The first is the progress notification issue as reported in #2621. MCP's progress notification mechanism is designed to provide real-time updates for long-running tasks. This user reports that while the progress notifications are successfully generated and visible in the browser network tab, the MCP inspector does not react to them until the tool ends with a final response. Additionally, in issue #625, another user reports that the "resources/list" method implementation is missing while the other methods work as expected. Lastly, a user complains in issue #106 about the need to have developer logs for failing MCP server invocations, as the displayed logs are not helpful.

RQ2: What is the extent of work involved in resolving issues affecting MCP server integration?

Motivation. In this RQ, our goal is to understand the extent of work involved in resolving the issues identified in RQ1. The official MCP repository currently provides no visible mechanism to prioritize or rank issues. The aim of this RQ is to measure the level of work required to resolve the various categories of issues by analyzing the pull requests that resolved the issues. With this, the maintainers of the MCP repository will be able to identify the areas of the repository that require more attention and prioritize them accordingly.

Approach. To answer this question, we adopt the use of code churn to measure the extent of work required to resolve the reported issues. Code churn has been widely adopted in empirical software engineering research as a metric for measuring developer effort involved in code changes [4]. For the purpose of this research, we define code churn as the sum of the lines of code added and deleted. We filter all closed issues from our dataset. Issues are closed for various reasons, including those that could not be reproduced by the maintainers. To focus only on issues that were closed after they were successfully resolved, we use the GitHub API to identify pull requests that are linked to those issues. First, we perform a search for all pull requests where each issue in our dataset was mentioned using GitHub's search API using the query - "repo:modelcontextprotocol/servers type:pr #issue_number". Second, using the GitHub's timeline API, we mine all "closed" events linked with each issue and cross-referenced PR links to identify issues that were resolved through merged pull requests. We identify that some issues were resolved by multiple PRs and factor that in our analysis. We observe that some PRs were explicitly mentioned in issue discussions as the solution to such issues without being directly linked. To capture such PRs, we implement regular expression patterns to identify such references in the discussion.

Next, we sum up the total number of lines of code added and the total number of lines deleted in the PR linked to each issue. Since some issues are resolved with multiple PRs, we sum up the metrics in all PRs associated with such issues. So, for example, if an issue was confirmed resolved after four PRs were merged, we sum up the metrics in all associated PRs. Then, we aggregate the metrics per issue category to calculate the average churn. **Out of all the**

Table 3: Code Churn Across Issue Categories

Issue Category	Total	Res.	Avg Churn
Connection and Communication	15	9	1143.5
File Path and File System	29	11	492.0
Data Validation and Type	19	10	402.7
Documentation and Tool Description	16	11	309.6
Configuration and Environment	17	9	143.5
Tool Invocation, Visibility, and Behaviour	18	5	95.7

issues in our dataset, 80 of them are closed, out of which 41 have associated PRs.

Results. Table 3 presents the results of our analysis. We only present results for categories that have at least five resolved issues. Our analysis reveals that **connection and communication issues require the highest amount of work to resolve (Avg churn: 1143.5)**. Trailing that is the **File Path and File System category (Avg. churn: 492.0)**, followed by Data Validation and Type Issues (Avg. churn: 402.7) and Documentation and Tool Description Issues (Avg. churn: 309.6). Furthermore, Configuration and Environment Issues and Tool Invocation, Visibility, and Behaviour Issues have average churn values of 143.5 and 95.7 respectively.

5 Discussion

We interpret our results and provide recommendations for MCP repository maintainers and contributors to guide them on how to mitigate these challenges.

Examining the Most Prevalent Challenges. The result in RQ1 shows that **file path and file system issues are the most prevalent ones (21.32%)**. Notably, this category highlights some security vulnerability concerns, as shown in issue #1845 where an AI agent is able to access files outside a clearly defined sandbox environment via symbolic links. This aligns with findings from Hasan et al. [5], who mentioned insecure file creation as one of the eight vulnerability patterns identified in MCP servers. Our work extends this by showing that these issues go beyond theoretical speculation and represent practical challenges developers face. **Data validation and type issues (13.97%)** expands on another finding of Hasan et al. [5] who listed input validation issues as another vulnerability pattern identified in MCP servers. Similarly, Tiwari et al. [13] identified a lack of runtime schema validation as one of the flaws identified in their study. **Tool invocation, visibility, and behaviour issues (13.24%)** are concerning because they impact the core idea of MCP, which is to have a seamless tool integration with LLMs.

We recommend a comprehensive review of the filesystem server to address the highlighted challenges. Also, we recommend the development of a reusable input validation library that is well-suited for the MCP ecosystem. Additionally, the MCP ecosystem would benefit from comprehensive tests across multiple real-world scenarios to mitigate the tool-call challenges.

The Need for a Prioritization Strategy. Our analysis in RQ2 shows a concerning detail about the resolution rate. While tool invocation is central to the core idea of MCP, only 27.78% of the issues have been resolved. Similarly, only 30% of start up issues have been resolved. This is concerning since such issues completely render an MCP server unusable, as they occur during the initialization phase of the MCP lifecycle[3]. We recommend that the repository maintainers implement a prioritization strategy based on these findings. Additionally, tool description issues are critical as wrong tool calls can be catastrophic, potentially causing irreversible damage.

6 Threats to Validity

Threats to Construct Validity consider the extent to which our analysis accurately reflects the theoretical constructs it is intended to assess. A threat to construct validity is our reliance on developer-reported GitHub issues, which may not fully represent all the challenges experienced by developers in practice. However, Github issues are a widely used and accepted data source in empirical software engineering research.

Threats to Internal Validity consider the researchers' bias and errors. First, our method of analysis is primarily manual. The subjective nature of manually categorizing the issues is a potential threat to the validity of our study. This subjectivity may affect the reproducibility of our categorization. To mitigate this risk, two authors independently categorized the GitHub issues and calculated the Cohen's Kappa coefficient, achieving a substantial agreement value of 0.68. Discrepancies and disagreements were resolved through discussion among the authors. Also, in our GitHub issues dataset, we applied a Python script to filter out non-reference MCP servers using keyword-based filtering. However, there is no assurance that all issues captured are indeed reference servers only. To mitigate this, two authors independently reviewed each issue in our dataset manually and discussed disagreements to reach a consensus. Additionally, percentage of issues resolved may not be a great metric as it is time-dependent. Hence, we focused on measuring code churn as the metric for evaluating the extent of work in resolving issues.

Threats to External Validity consider the generalizability of findings. In our study, we focused only on reference servers, which may not fully represent the wider range of MCP servers. This approach was implemented to mitigate the introduction of variations that may arise from third-party developer integration methods. Future work can go further to analyze community-developed MCP servers.

7 Acknowledgements

The icons used in figure 1 are freely provided by Freepik (https://www.flaticon.com)

8 Conclusion and Future Work

In this study, we presented a comprehensive analysis of developer-reported MCP issues, categorized them, and highlighted the most prevalent ones. Through manual inspection of the 136 reference server-related issues from the official MCP repository, we identified significant patterns in the issue types, we proposed resolution strategies, and evaluated the extent of work required to resolve each issue category using average code churn. Our findings revealed the need to prioritize file path and file system issues, tool invocation-related issues, start up and initialization issues, and documentation/tool description issues. While file system issues are the most common,

[3] https://modelcontextprotocol.io/specification/2025-06-18/basic/lifecycle

communication issues require the most effort to resolve. Based on our findings, we made several recommendations. Notably, the need to develop an issue resolution prioritization strategy, as our findings show that the most critical issues have very low resolution rates. Our study provides the foundation to develop such a prioritization strategy. Future work can compare these findings with community servers to identify similarities and differences in issue types and their prevalence.

References

[1] Model Context Protocol 2025. *Modelcontextprotocol/Servers*. Model Context Protocol. https://github.com/modelcontextprotocol/servers

[2] Joseph L. Fleiss and Jacob Cohen. 1973. The Equivalence of Weighted Kappa and the Intraclass Correlation Coefficient as Measures of Reliability. *Educational and Psychological Measurement* 33, 3 (1973), 613–619. doi:10.1177/001316447303300309

[3] Hechuan Guo, Yongle Hao, Yue Zhang, Minghui Xu, Peizhuo Lv, Jiezhi Chen, and Xiuzhen Cheng. 2025. *A Measurement Study of Model Context Protocol Ecosystem*. arXiv:2509.25292 [cs] doi:10.48550/arXiv.2509.25292

[4] Gregory A. Hall and John C. Munson. 2000. Software Evolution: Code Delta and Code Churn. *Journal of Systems and Software* 54, 2 (2000), 111–118. doi:10.1016/S0164-1212(00)00031-5

[5] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. 2025. *Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers*. arXiv:2506.13538 [cs] doi:10.48550/arXiv.2506.13538

[6] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VtmBAGCN7o

[7] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. *Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions*. arXiv:2503.23278 [cs] doi:10.48550/arXiv.2503.23278

[8] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. 2023. *Augmented Language Models: A Survey*. arXiv:2302.07842 [cs] doi:10.48550/arXiv.2302.07842

[9] Oluwatoyosi Oyelayo, Samuel Abedu, SayedHassan Khatoonabadi, and Emad Shihab. 2025. *Developer Challenges in the Adoption of Model Context Protocol (MCP)*. https://zenodo.org/records/17445547

[10] Haochen Pan, Ryan Chard, Reid Mello, Christopher Grams, Tanjin He, Alexander Brace, Owen Price Skelly, Will Engler, Hayden Holbrook, Song Young Oh, Maxime Gonthier, Michael Papka, Ben Blaiszik, Kyle Chard, and Ian Foster. 2025. *Experiences with Model Context Protocol Servers for Science and High Performance Computing*. arXiv:2508.18489 [cs] doi:10.48550/arXiv.2508.18489

[11] Minh Vu Phong, Tam The Nguyen, Hung Viet Pham, and Tung Thanh Nguyen. 2015. Mining User Opinions in Mobile App Reviews: A Keyword-Based Approach (T). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2015-11). 749–759. doi:10.1109/ASE.2015.85

[12] Weixiong Rao, Lei Chen, Pan Hui, and Sasu Tarkoma. 2012. MOVE: A Large Scale Keyword-Based Content Filtering and Dissemination System. In *2012 IEEE 32nd International Conference on Distributed Computing Systems* (2012-06). 445–454. doi:10.1109/ICDCS.2012.32

[13] Aditi Tiwari, Akshit Bhalla, and Darshan Prasad. 2025. *Model Context Protocol for Vision Systems: Audit, Security, and Protocol Extensions*. arXiv:2509.22814 [cs] doi:10.48550/arXiv.2509.22814

[14] Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, and Eugene Siow. 2025. *MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers*. arXiv:2508.20453 [cs] doi:10.48550/arXiv.2508.20453

[15] Bo Wen, Chen Wang, Qiwei Han, Raquel Norel, Julia Liu, Thaddeus Stappenbeck, and Jeffrey L. Rogers. 2025. *Voice-Based AI Agents: Filling the Economic Gaps in Digital Health Delivery*. arXiv:2507.16229 [cs] doi:10.48550/arXiv.2507.16229

[16] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. 2024. TravelPlanner: a benchmark for real-world planning with language agents. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML'24)*. JMLR.org, Article 2246, 24 pages.

[17] Lingfeng Zeng, Fangqi Lou, Zixuan Wang, Jiajie Xu, Jinyi Niu, Mengping Li, Yifan Dong, Qi Qi, Wei Zhang, Ziwei Yang, Jun Han, Ruilun Feng, Ruiqi Hu, Lejie Zhang, Zhengbo Feng, Yicheng Ren, Xin Guo, Zhaowei Liu, Dongpo Cheng, Weige Cai, and Liwen Zhang. 2025. *FinGAIA: A Chinese Benchmark for AI Agents in Real-World Financial Domain*. arXiv:2507.17186 [cs] doi:10.48550/arXiv.2507.17186